

Exercices de révision

Exercice 1

Comparateur de Stabilité Météorologique entre Deux Villes

Écrivez un programme qui aide un utilisateur à choisir entre deux villes en fonction des conditions météorologiques. Le service météorologique mesure la température à trois moments différents de la journée : le matin, à midi, et le soir. L'utilisateur préfère se rendre dans la ville où la variation de température entre n'importe lesquels de ces trois moments ne dépasse pas 5 degrés Celsius. Le programme doit comparer les températures des deux villes et indiquer laquelle est préférable selon ce critère de stabilité thermique. Utiliser des tableaux.

Exercice 2

Écrire un programme pour déterminer le point de rencontre de deux "rivières numériques".

Principe des rivières numériques : Une rivière numérique est une séquence de nombres où chaque nombre est suivi par le même nombre additionné de la somme de ses chiffres. Par exemple, dans une telle séquence, le nombre 123 est suivi par 129 (car $1 + 2 + 3 = 6$), qui est à son tour suivi par 141.

Exemples :

- La rivière 7 commence par la séquence {7, 14, 19, 29, 40, 44, 52, ...}
- La rivière 471 commence par la séquence {471, 483, 498, 519, ...}

Les rivières numériques peuvent se rencontrer, c'est-à-dire partager les mêmes valeurs. Par exemple, la rivière 32 rencontre la rivière 47 au nombre 47, tandis que la rivière 471 rencontre la rivière 480 au nombre 519.

- Le programme doit accepter deux nombres entiers, K1 et K2, qui représentent les points de départ de deux rivières numériques distinctes.
- Faire avancer chaque rivière tant que leurs chemins ne se rencontrent pas.
- Afficher le premier point de rencontre des deux rivières numériques.

Vous pouvez utiliser la fonction suivante qui permet de calculer la somme des digits d'un nombre :

```
1. int sum_of_digits(int n)
2. {
3.     int sum = 0;
4.     while (n > 0)
5.     {
6.         sum += n % 10;
7.         n /= 10;
8.     }
9.     return sum;
10. }
```

Exercice 3

Considérez que vous avez la fonction ci-dessous.

Ecrire le *main* et utiliser cette fonction afin qu'un rectangle de longueur 4 et de hauteur 3 s'affiche.

```
void afficheRectangle(int l, int lg)
{
    printf("je vais afficher un rectangle de taille %d x %d \n", l, lg);
    for (int i = 0; i < l; i++)
    {
        for (int j = 0; j < lg; j++)
        {
            printf("o");
        }
        printf("\n");
    }
}
```

Exercice 4

Voici un code qui contient des bugs.

Expliquez ces bugs et corrigez-les.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {

    int tentatives = 10;

    int devinette = -1;
    // Initialisation du générateur de nombres aléatoires
    srand(time(NULL));
    int nombreSecret = rand() % 100 + 1;

    printf("Devinez le nombre secret entre 1 et 100. Vous avez %d tentatives.\n", tentatives);

    while ( (devinette != nombreSecret) || (tentatives > 0) ) {
        printf("Entrez votre devinette: ");
        scanf_s("%d", &devinette);

        if (devinette < 1 || devinette > 100) {
            printf("Votre devinette doit être entre 1 et 100!\n");
            continue;
        }

        if (devinette == nombreSecret) {
            printf("Félicitations ! Vous avez trouvé le nombre secret.\n");
        }
        else if (devinette < nombreSecret) {
            printf("Le nombre secret est plus grand.\n");
        }
        else {
            printf("Le nombre secret est plus petit.\n");
        }

        tentatives--;
        printf("Il vous reste %d tentatives.\n", tentatives);
    }

    if (tentatives == 0) {
        printf("Désolé, vous avez utilisé toutes vos tentatives. Le nombre secret était %d.\n", nombreSecret);
    }

    return 0;
}
```

Exercise 5

Réaliser une animation de chute de neige dans le terminal. L'animation repose sur une grille de caractères représentant l'écran, dans laquelle des flocons apparaissent, tombent progressivement vers le bas, et sont redessinés à chaque instant.

L'écran est modélisé par une grille rectangulaire de taille fixe :

- une largeur de 80 caractères,
- une hauteur de 25 lignes.

Chaque case de la grille peut contenir:

- soit un flocon de neige (*),
- soit du vide (espace).

Le programme fonctionne en boucle infinie et réalise les étapes suivantes :

1. Afficher l'état actuel de la grille dans le terminal.
2. Mettre à jour la position des flocons :
 - un flocon peut tomber d'une ligne s'il n'y a rien en dessous,
 - les flocons se déplacent uniquement vers le bas.
3. Attendre un court délai afin de rendre l'animation visible.
4. Recommencer depuis le début.

L'écran est réaffiché en permanence, donnant l'illusion d'une chute de neige continue.

[illegible]

Exercise 6

Écrire une fonction récursive (= une fonction qui s'appelle elle-même) qui affiche un compte à rebours de n à 0, puis affiche "Boom!".

Exercice 7

Bataille Navale simplifiée :

Créer un petit jeu où l'utilisateur doit deviner la position d'un navire sur une grille 5x5.

Exercice 8

Simuler la réservation de sièges dans une salle de concert. L'utilisateur peut choisir des sièges à réserver et le programme doit afficher les sièges disponibles et occupés. L'utilisateur peut faire plusieurs réservations, c'est à lui de dire quand il a fini.